# PETSc Course

Victor Eijkhout

2026 COE 379L / CSE 392

# To set the stage

Developing parallel, nontrivial PDE solvers that deliver high performance is still difficult and requires months (or even years) of concentrated effort. PETSc is a toolkit that can ease these diffculties and reduce the development time, but it is not black-box PDE solver, nor a silver bullet.

Barry Smith

# More specifically. . .

Portable Extendable Toolkit for Scientific Computations

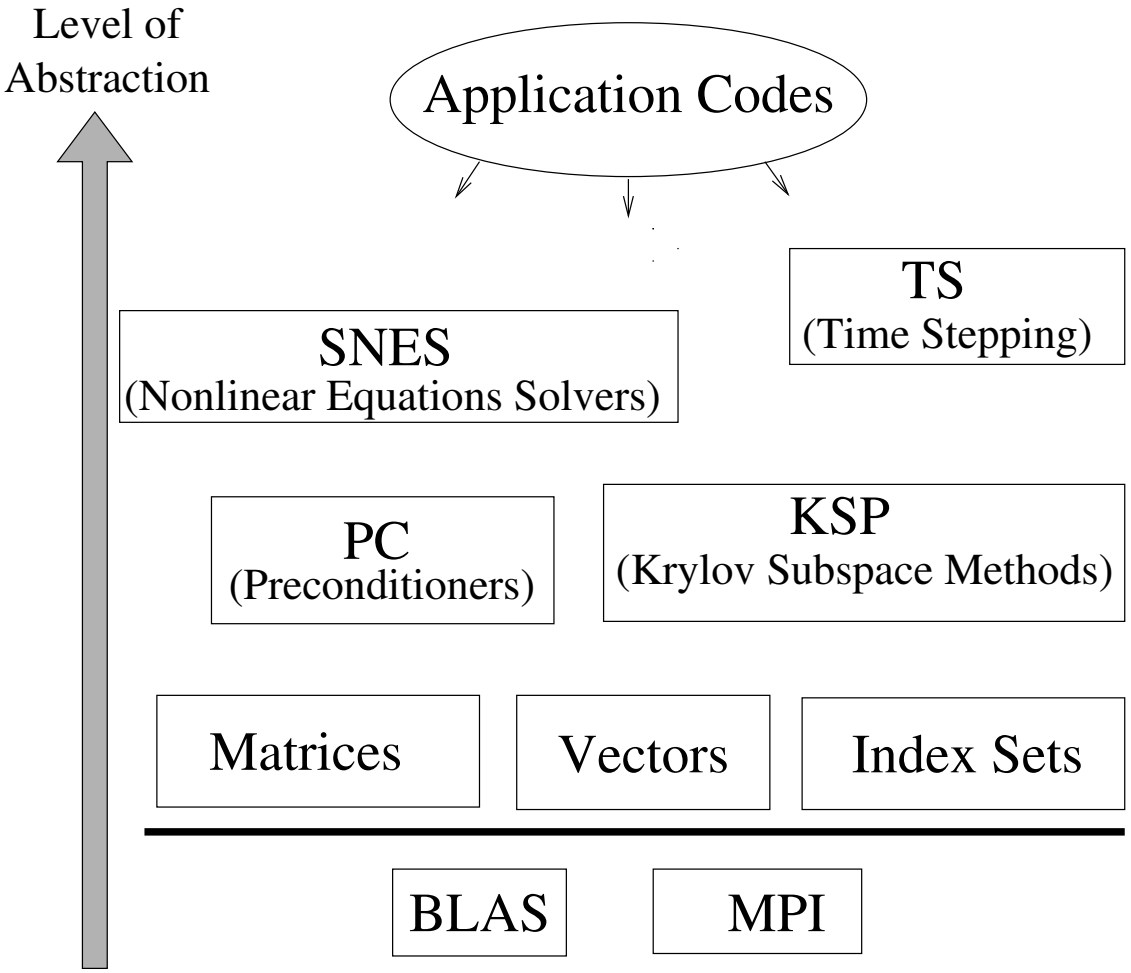
- Scientific Computations: parallel linear algebra, in particular linear and nonlinear solvers
- Toolkit: Contains high level solvers, but also the low level tools to roll your own.
- Portable: Available on many platforms, basically anything that has MPI

Why use it? It's big, powerful, well supported.

# What is in PETSc?

- Linear algebra data structures, all serial/parallel
- Linear system solvers (sparse/dense, iterative/direct)
- Nonlinear system solvers
- Optimization: `TAO` (used to be separate library)
- Eigenvalue solvers (including SVD): `SLEPc` (integrated)
- Tools for distributed matrices
- Support for profiling, debugging, graphical output

# Structure of a PETSc application



# Hierarchy of tools

## Parallel Numerical Components of PETSc

| Nonlinear Solvers | | |
|---|---|---|
| Newton−based Methods | | Other |
| Line Search | Trust Region | |

| Time Steppers | | | |
|---|---|---|---|
| Euler | Backward Euler | Pseudo−Time Stepping | Other |

| Krylov Subspace Methods | | | | | | | |
|---|---|---|---|---|---|---|---|
| GMRES | CG | CGS | Bi−CG−Stab | TFQMR | Richardson | Chebychev | Other |

| Preconditioners | | | | | | |
|---|---|---|---|---|---|---|
| Additive Schwarz | Block Jacobi | Jacobi | ILU | ICC | LU (sequential only) | Other |

| Matrices | | | | |
|---|---|---|---|---|
| Compressed Sparse Row (AIJ) | Block Compressed Sparse Row (BAIJ) | Block Diagonal (BDiag) | Dense | Other |

| Vectors |
|---|

| Index Sets | | | |
|---|---|---|---|
| Indices | Block Indices | Stride | Other |

# Documentation and help

- Web page: `https://petsc.org/`
- Documentation (pdf/html): `https://petsc.org/release/docs/`
- Follow-up to this tutorial: eijkhout@tacc.utexas.edu
- PETSc on your local cluster: ask your local support
- General questions about PETSc: petsc-maint@mcs.anl.gov
- Example codes, found online, and in `$PETSC_DIR/src/mat/examples` et cetera
- Sometimes consult include files, for instance `$PETSC_DIR/include/petscmat.h`

# External packages

PETSc does not do everything, but it interfaces to other software:

- Dense linear algebra: `Scalapack`, `Plapack`, `Elemental`
- Grid partitioning software: `ParMetis`, `Jostle`, `Chaco`, `Party`
- ODE solvers: `PVODE`

# PETSc and parallelism

PETSc is layered on top of MPI

- MPI has basic tools: send elementary datatypes between processors
- PETSc has intermediate tools:
  insert matrix element in arbitrary location,
  do parallel matrix-vector product
- Transparent: same code works sequential and parallel.
  (Some objects explicitly declared `Seq/MPI`)
- $\Rightarrow$ you do not need to know much MPI when you use PETSc
- All objects in Petsc are defined on a communicator;
  can only interact if on the same communicator
- No OpenMP used in the library:
  user can use shared memory programming.
- Likewise, threading is kept outside of PETSc code.
- ~~Limited Graphics Processing Unit (GPU) support; know what you're doing!~~

# Object oriented design

Petsc uses objects: vector, matrix, linear solver, nonlinear solver

Overloading:

```
MATMult(A,x,y); // y <- A x
```

same for sequential, parallel, dense, sparse, FFT

# Data hiding

To support this uniform interface, the implementation is hidden:

```
MatSetValue(A,i,j,v,INSERT_VALUES); // A[i,j] <- v
```

There are some direct access routines, but most of the time you don't need them.

(And don't worry about function call overhead.)

# 1. `Vec` datatype: vectors

# Create calls

Everything in PETSc is an object, with create and destroy calls:

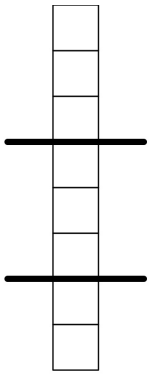
```
VecCreate(MPI_Comm comm,Vec *v);
VecDestroy(Vec *v);

Vec V;
VecCreate(MPI_COMM_WORLD,&V);
VecDestroy(&V);
```

# Parallel layout up to PETSc

```
VecSetSizes(Vec v, int m, int M);
```

Local size can be specified as *PETSC_DECIDE*.



VecSetSizes(V,PETSC_DECIDE,8)

VecSetSizes(V,PETSC_DECIDE,8)

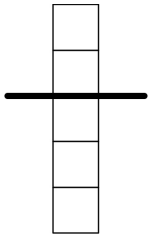
VecSetSizes(V,PETSC_DECIDE,8)

# Parallel layout specified

Local or global size in

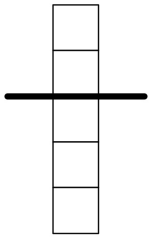
```
1 VecSetSizes(Vec v, int m, int M);
```

Global size can be specified as *PETSC_DECIDE*.



VecSetSizes(V,2,5)

VecSetSizes(V,3,5)



VecSetSizes(V,2,PETSC_DECIDE)

VecSetSizes(V,3,PETSC_DECIDE)

# Setting values

Set vector to constant value:

```
VecSet(Vec x,PetscScalar value);
```

Set individual elements (global indexing!):

```
VecSetValue
   (Vec x,int row,PetscScalar value,
    InsertMode mode);

i = 1; v = 3.14;
VecSetValue(x,i,v,INSERT_VALUES);

call VecSetValue(x,i,v,INSERT_VALUES,e)
```

The other insertmode is *ADD_VALUES*.

# Basic operations

```
VecAXPY(Vec y,PetscScalar a,Vec x);    /* y <- y + a x */
VecAYPX(Vec y,PetscScalar a,Vec x);    /* y <- a y + x */
VecScale(Vec x, PetscScalar a);
VecDot(Vec x, Vec y, PetscScalar *r); /* several variants */
VecMDot(Vec x,int n,Vec y[],PetscScalar *r);
VecNorm(Vec x,NormType type, PetscReal *r);
VecSum(Vec x, PetscScalar *r);
VecCopy(Vec x, Vec y);
VecSwap(Vec x, Vec y);
VecPointwiseMult(Vec w,Vec x,Vec y);
VecPointwiseDivide(Vec w,Vec x,Vec y);
VecMAXPY(Vec y,int n, PetscScalar *a, Vec x[]);
VecMax(Vec x, int *idx, double *r);
VecMin(Vec x, int *idx, double *r);
VecAbs(Vec x);
VecReciprocal(Vec x);
VecShift(Vec x,PetscScalar s);
```

# 2. `Mat` Datatype: matrix

# Matrix creation

The usual create/destroy calls:

```
MatCreate(MPI_Comm comm,Mat *A)
MatDestroy(Mat *A)
```

Several more aspects to creation:

```
MatSetType(A,MATSEQAIJ) /* or MATMPIAIJ or MATAIJ */
MatSetSizes(Mat A,int m,int n,int M,int N)
MatSeqAIJSetPreallocation /* more about this later*/
  (Mat B,PetscInt nz,const PetscInt nnz[])
```

Local or global size can be *PETSC_DECIDE* (as in the vector case)

# Matrix operations

Main operations are matrix-vector:

```
MatMult(Mat A,Vec in,Vec out);
MatMultAdd
MatMultTranspose
MatMultTransposeAdd
```

Simple operations on matrices:

```
MatNorm

MatScale
MatDiagonalScale
```

# Shell matrices

What if the matrix is a user-supplied operator, and not stored?

```
MatSetType(A,MATSHELL); /* or */
MatCreateShell(MPI Comm comm,
    int m,int n,int M,int N,void *ctx,Mat *mat);

PetscErrorCode UserMult(Mat mat,Vec x,Vec y);

MatShellSetOperation(Mat mat,MatOperation MATOP_MULT,
  (void(*)(void)) PetscErrorCode (*UserMult)(Mat,Vec,Vec));
```

Inside iterative solvers, PETSc calls `MatMult(A,x,y)`:
no difference between stored matrices and shell matrices

# 3. `KSP` & `PC`: Iterative solvers

# What are iterative solvers?

Solving a linear system $Ax = b$ with Gaussian elimination can take lots of time/memory.

Alternative: iterative solvers use successive approximations of the solution:

- Convergence not always guaranteed
- Possibly much faster / less memory
- Basic operation: $y \leftarrow Ax$ executed once per iteration
- Also needed: preconditioner $B \approx A^{-1}$

# Iterative solver basics

- *KSP* object: solver
- set linear system operator
- solve with rhs/sol vector
- this is a default setup

```
KSPCreate(comm,&solver); KSPDestroy(solver);
// set Amat and Pmat
KSPSetOperators(solver,A,B); // usually: A,A
// solve
KSPSolve(solver,rhs,sol);
```

Optional: `KSPSetUp(solver)`

# Exercise 1 `ksp`

File `ksp.c` / `ksp.F90` contains the solution of a (possibly nonsymmetric) linear system.

Compile the code and run it. Now experiment with commandline options. Make notes on your choices and their outcomes.

- The code has two custom commandline switch:
  - `-n 123` set the domain size to 123 and therefore the matrix size to $123^2$.
  - `-unsymmetry 456` adds a convection-like term to the matrix, making it unsymmetric. The numerical value is the actual element size that is set in the matrix.
- What is the default solver in the code? Run with `-ksp_view`
- Print out the matrix for a small size with `-mat_view`.
- Now out different solvers for different matrix sizes and amounts of unsymmetry. See the instructions in the code.